

Reviewer Pack — Minimal p-Adic Valuation Degree and Circuit Monotone Width I...

Entry b8fcad7929fc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 13:07:19 UTC

Minimal p-Adic Valuation Degree and Circuit Monotone Width Inequality

Entry ID: b8fcad7929fc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 13:07:19 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis (Valuation Theory) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every Boolean circuit C with n inputs, the minimal p-adic valuation degree of its gates, defined as the smallest p-adic valuation $v(p)$ such that all gate outputs are expressible in terms of p-adjacent elements with valuation $\leq v(p)$, satisfies the inequality: $w_{\text{mon}}(C) = \Theta(v(p)^{2n})$

Rationale (proposer's reasoning):

p-adic analysis provides a non-Archimedean metric, which can potentially expose non-trivial complexity-theoretic properties that are not visible in Archimedean settings. The valuation degree measures the level of precision required for efficient computation, and its relationship with circuit monotone width could reveal insights into computational complexities.

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ad66b40a5f48548c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the monotone width $w_{\text{mon}}(C)$ of each Boolean circuit C meets $w_{\text{mon}}(C) \leq 1.5 * v(p)^{2n}$, and falsified if any circuit has $w_{\text{mon}}(C) > 1.5 * v(p)^{2n}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - $v(p\text{-adic valuation AND circuit monotone width})$ - Boolean circuit complexity AND $p\text{-adic valuation degree INCLUSIVE}$ - $p\text{-adic analysis AND Boolean circuits WITHIN inequality } v(p)^{2n}$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_circuit(n):
    if n == 1:
        return ['0', '1']
    left = generate_circuit(n // 2)
    right = generate_circuit(n - n // 2)
    return [f'({l} AND {r})' for l in left] + [f'({l} OR {r})' for r in right]

def evaluate_circuit(circuit):
    if isinstance(circuit, str):
        return circuit
    else:
        results = [evaluate_circuit(subcircuit) for subcircuit in circuit]
        return '1' if any(result == '1' for result in results) else '0'

def p_adic_valuation(circuit):
    if isinstance(circuit, str):
        return 0
    else:
        return max(p_adic_valuation(subcircuit) for subcircuit in circuit)

def monotone_width(circuit):
    if isinstance(circuit, str):
        return 1
    else:
        left_width = monotone_width(circuit[0])
        right_width = monotone_width(circuit[2])
        return max(left_width + 1, right_width + 1)

def run_trial(seed: int) -> dict:
    random.seed(seed)
```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    circuit = generate_circuit(n)
    v_p = p_adic_valuation(circuit)
    w_mon = monotone_width(circuit)
    results.append((v_p, w_mon))

    if len(results) >= 30:
        break

total_v_p = sum(v_p for v_p, _ in results)
total_w_mon = sum(w_mon for _, w_mon in results)
mean_v_p = total_v_p / len(results)
mean_w_mon = total_w_mon / len(results)

conjecture_holds = all(w_mon <= 1.5 * v_p**2 * n for v_p, w_mon, n in zip(results, results, n_values))
counterexample = "" if conjecture_holds else "n/a"

return {
    "metric_name": "Monotone Width vs p-Adic Valuation",
    "metric_value": mean_w_mon,
    "instances_tested": len(results),
    "n_max": max(n for _, _, n in results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='n/a' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e62a38ae.py", line 83, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e62a38ae.py", line 52, in run_trial
    circuit = generate_circuit(n)
              ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e62a38ae.py", line 21, in generate_circ
    left = generate_circuit(n // 2)
           ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e62a38ae.py", line 23, in generate_circ
    return [f'({l} AND {r})' for l in left] + [f'({l} OR {r})' for r in right]
           ^
NameError: name 'r' is not defined. Did you mean: 're'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE
Critic reasoning:
skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE
Reasoning:
The test code crashed before producing data, which prevents us from verifying the conjecture’s conditions. | next: Debug the test code to ensure it runs successfully and produces the required data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	24236
2	preregistration	ollama_remote	glm4:latest	0	0	9648
3	novelty	ollama_remote	glm4:latest	0	0	8332
4	novelty	ollama_remote	glm4:latest	0	0	9931
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15538
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14782
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11578
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10325
9	judge	ollama_remote	glm4:latest	0	0	11576

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 115946 ms total latency. Provider mix: {‘ollama_remote’: 9}
(full prompt+response transcripts available in research/audit/b8fcad7929fc.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b8fcad7929fc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/b8fcad7929fc.tar.gz` (if generated)